



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)  
'walls': list(self.walls)  
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

- **State Space:** The set of all possible physical configurations or valid locations in the environment graph/grid. It is bounded by the grid dimensions minus obstacles.
- **Search Tree:** An explicit mathematical tree constructed during search where the root node is the start state, branches represent possible actions, and paths represent action sequences. A single physical state from the state space can appear multiple times across different paths/branches in a search tree.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

- **Problem Solved:** It converts a **Tree Search** into a **Graph Search** by keeping track of expanded/visited states to eliminate redundant paths and cyclic loops.
- **Without it:** The DFS agent would get trapped in an **infinite loop**, repeatedly moving back and forth between adjacent unvisited grid cells

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

- **BFS Optimality:** BFS expands nodes layer-by-layer in increasing order of path depth. Because every step cost in an unweighted grid environment is uniform, the first time BFS reaches a target food state, it is guaranteed to have found the shortest path.
- **DFS Suboptimality:** DFS expands nodes down a single branch to maximum depth before backtracking. It follows a specific child expansion order (e.g., Up, Down, Left, Right), causing it to wander along deep, arbitrary paths until it happens to hit a wall or food, resulting in long, winding routes.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

- **BFS Memory Bottleneck:** BFS has an exponential space complexity of $O(b^d)$, where b is the branching factor and d is the depth of the optimal solution. On a 1000×1000 grid, it stores every generated node at the current depth in the frontier queue simultaneously, quickly causing memory exhaustion.
- **DFS Memory Efficiency:** Graph DFS requires $O(V)$ space to store the visited set (up to 1,000,000 cells for a 1000×1000 grid). Standard Tree DFS has a linear space complexity of $O(bm)$, storing only the active path from the root to the current node along with unexpanded siblings, making it far less memory-intensive than BFS.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING